Release plan — Mongoose 1.0.33 and mongoose-plugins 1.0.46

!!! warning "Historical plan — superseded" The RC includes PR #76 as well as #69. Use the RC release procedure and 1.0.33-rc1 notes. The stable-release steps and scope below are retained as historical context, not current release instructions.

Status: draft, 2026-10-24. Covers the thread-safety, lifecycle and admin-REST fixes found by review on 2026-10-23/24. Integration branch for testing: `integration/next-release` (draft PR #69, https://github.com/telaminai/mongoose/pull/69).

1. What is in the release

Mongoose (telaminai/mongoose) — 13 PRs: three stacks, three PRs on the agent stack, one docs PR

Stack	PRs (merge order)	Issues	What it fixes
Dispatch	#52 → #63 → #64	#51, #54, #59	Concurrent publishers lost 54–60 % of events (HandlerPipe); a queue removed during dispatch threw out of `publish()` and exited the process; pooled events were returned to the pool while in use and the pool ran dry; the event-log cache grew for ever after start.
Lifecycle	#61 → #65	#56, #60 (items 2–4)	Server stop never stopped or tore down processors or feeds; duplicate-name races; startup waited for ever; feeds spun on dead queues at shutdown.
Processor agent	#62 → #66 → #67 → #68	#55, #57, #58, #60 (item 1)	A processor stopped and re-added got no events; one processor's failure exited the JVM; single-producer control queues fed from several threads; services delivered twice.
On #68 (independent of each other)	#73	#53	A retry after one processor threw re-delivered the event to every processor on the queue (a repeated side effect, e.g. an order sent twice) and leaked the processor context; a `VirtualMachineError` from a handler was retried and swallowed.
	#72	#71	A normal `stopProcessor` left the processor's timers (they fired into it), replay and audit bindings; a `stop()` that threw skipped `tearDown()`; a stop made while the processor's add was still queued did nothing.
	#74	#46	Audit start/stop: the agent thread could wait 5 s on its own caller (lock held across the hand-off); a stopped processor or a closed group was reported as recording; refusals were thrown to the transport instead of replied.
Docs	#70	#50	The deployment guide presented clustering, state replication, failover and tracing as supported.

Integration commits (on `integration/next-release`; to land as one follow-up PR once everything above is on main):

- **#65 × #66**: a processor dropped after a failed start releases its reserved name, and the boot fail-fast check reads #65's lifecycle state instead of #66's `started` flag, which #65 replaced. Without it the two stacks merge but **do not compile together**. Textual conflict: one line pair in `MongooseServerConfig.Builder`.

- **#72 × #74**: both add `MongooseServer.processorStopped`, and the merged agent called it twice. One hook: the audit capture forgets the stopped instance (#74), then a capture still recording is ended and closed (#72).
- **#63 × #73**: both rewrote the queue agent's dispatch loop. #73's per-processor retry sits inside #63's frame (the pooled reference released in a `finally`).
- **#65 × #74**: at server stop `stopAll` forgets each instance before ending its capture, since #74's stop refuses a closed group.

All open issues found by the reviews are fixed. Not covered: downstream client code that has not yet moved to Mongoose.

mongoose-plugins (telaminai/mongoose-plugins)

PR	What it fixes
#40 svc-admin-rest	`POST /admin` returned 500 for every request (Javalin 6 has no JSON mapper without jackson-databind); 400 on bad bodies, 404 on unknown commands, 503 without a registry; one reply per request.
#41 svc-admin-web	A body that failed to parse ran the command with no arguments; accept `arguments` and `args`; 400 for bad login bodies; jackson version aligned.
(adoption PR, after Mongoose 1.0.33 is released)	Bump to Mongoose 1.0.33 and fluxtion-bom ≥ 1.0.76; schema/version bookkeeping; four connectors move to `startComplete()`.

2. Review and test status

Every PR was reviewed independently (verdicts: approve with nits, no blockers); every review finding has been fixed on its branch with a test that fails without the fix. The dispatch stack merges cleanly with either of the others; the other combinations need the integration commits above. Results:

Where	Tests	Result
`integration/next-release` (all thirteen PRs + integration commits), `mvn test`, 3 runs	473 (9 skipped)	0 failures, 0 errors in each run (~1 m 42 s)
GitHub CI (`main.yml`, 2-core runner) on every PR branch and on the integration branch		green
Processor-agent stack top (#68), 3 runs	384 (9 skipped)	0 failures
#62 / #66 / #67, one run each	355 / 370 / 377	0 failures
Integration test `processorDroppedAfterAFailedRuntimeStartReleasesItsName`	1	fails without the integration commit, passes with it
#73 (#53), #72 (#71), #74 (#46): new tests	10 / 6 / 7	each fix's tests fail on the code before it (except where noted in the PR: #74's load test passes since #47 already fixed the main path)
#65 additions: `ServerStopEndsAuditCaptureTest`, `stopWithAFeedThatOutrunsASlowProcessorCompletes`	2	the first fails without the change; the second pins that stop completes with a full processor queue
`publishersInALoopDoNotStarveARemoveOrAnAdd` (#52, after CI)	1	without the fix an add waited 39.9 s behind 4 looping publishers

The skipped tests are skipped on `main` as well.

3. Release steps

1. **Test the integration branch** `integration/next-release` (CI runs `mvn package` with tests on every push). It is the exact combination of all thirteen PRs plus the integration commits.
2. **Merge to main in stack order**, retargeting each PR to `main` once the one below it has merged (GitHub retargets automatically when a base branch is deleted after merge):
   - Dispatch: #52 → #63 → #64.
   - Lifecycle: #61 → #65.
   - Processor agent: #62 → #66 → #67 → #68, then #73, #72 and #74 (each based on #68, in any order).
   - Docs: #70 (any time).
   - The integration-fix PR last (it needs all of the above).

   After each stack, `main.yml` CI must be green.
3. **Release Mongoose 1.0.33**: push `main` to the `release` branch. `release.yml` runs `mvn -P release release:prepare release:perform` (tag `v1.0.33`, publish to Maven Central, next version 1.0.34-SNAPSHOT) and merges release → main → develop. `mongoose-browser-api` is not a module of the root pom and is not released.
4. **mongoose-plugins**: merge #40 and #41 (independent of Mongoose; they build on 1.0.29). Then the adoption PR:
   - `mongoose.version` → 1.0.33; `fluxtion.bom.version` → ≥ 1.0.76 (Mongoose ≥ 1.0.32 needs fluxtion 1.1.x; plugins today resolves fluxtion-runtime 1.0.15 — a latent `NoSuchMethodError` on admin lambda commands);
   - `plugin-index.json`: core built-ins' `sourceVersion` → 1.0.33;
   - regenerate `expected-schema.json` (new `retainEventLogAfterStart`) and add its doc override;
   - file, chronicle, aeron and multicast connectors: `dispatchCachedEventLog()` → `startComplete()` (otherwise their event log keeps growing);
   - optionally bump `example/admin-web-example` (pins Mongoose 1.0.28). Run the full reactor **with tests** (`./mvnw test`) — plugins CI runs `package -DskipTests` and its release skips tests, so the schema gates only run locally.
5. **Release mongoose-plugins 1.0.46** via its release branch.

4. Behaviour changes (release notes)

The control-plane changes of PR #76 are not part of 1.0.33. Several of them supersede notes below; see Release notes — unreleased.

Mongoose 1.0.33

- **Publishing**: publishes are serialised per publisher. A subscriber whose queue is full now stalls every publisher on that publisher for up to ~10 ms per event (the existing spin). Adding/removing a subscriber waits only for the dispatch in flight: publishers hold back while one is waiting, so a busy publisher cannot starve it (CI caught this: Java monitors are not fair). Items dropped on a full queue are counted (`droppedCount()`).
- **Pooling**: the last `releaseReference()` returns the object to the pool; a release below zero throws without decrementing; `removeFromPool()`/`cache()` detach an object for good.
- **Event-log cache**: with `cacheEventLog`, publishes after `startComplete` are no longer logged (`setRetainEventLogAfterStart(true)` — also on `HandlerPipe`/`HandlerPipeConfig` — restores the old

unbounded log). Sources must call `EventToQueuePublisher.startComplete()`; a source still calling `dispatchCachedEventLog()` at start keeps the old growth. A processor added at runtime no longer sees post-start history.

- **Stop**: `stop()` stops feeds, drains queued events for up to `shutdownDrainTimeoutMillis` (default 2 s), stops and tears down processors on their own threads, then services; it is idempotent; called on an agent thread it returns before the shutdown completes (`awaitStopped()` waits); concurrent callers wait.
- **Stop during synchronous startup**: stop now waits for an in-progress synchronous startup step to return before final cleanup completes. A service whose `start()` blocks and ignores `stop()` delays server stop. Give blocking startup operations a timeout or a cooperative cancellation path.
- **Startup cancellation**: a service in progress when stop arrives may receive `stop()` twice: first as the cancellation signal, then during final cleanup. Services must tolerate the second call.
- **Start**: fails after `startupTimeoutMillis` (default 60 s), or at once if a group closes or its thread dies during start; throws if called twice or after `stop()`.
- **Names**: a duplicate processor name is refused, and in YAML config now **fails boot**.
- **Shared sources**: a source is told to unsubscribe a key only when the last group reading it leaves; a source whose unsubscribe throws is reported and the group carries on.
- **Dropped processors**: a processor dropped after a failed start is torn down, its subscriptions released, its timers cancelled, its replay/audit bindings detached and its name freed for reuse. (A processor stopped normally with `stopProcessor` still keeps its timers and recorder/audit bindings, as before; not changed in this release.)
- **Services across groups**: `registerService`/`removeService` try every group; a stalled group is reported instead of throwing, and the service stays registered.
- **Retries (#53)**: a retry re-delivers only to the processor that threw, then the ones after it; each failing processor has its own `RetryPolicy` attempts, and one whose retries run out is skipped instead of the rest of the queue. A `VirtualMachineError` from a handler reaches the agent's error handler and is not retried. Recording marks only the failing processor's input as Failed. A custom `EventToInvokeStrategy` keeps whole-dispatch retry unless it implements the new `failedPosition()`/`processEventFrom(...)`.
- **Stopping a processor (#72)**: releases everything (timers cancelled, replay and audit bindings released, a capture still recording is ended); `tearDown()` runs even if `stop()` throws; a stop made just after the add is honoured. The name is free again once the group has processed the stop: an immediate re-add may be refused for an instant.
- **Audit start/stop (#74)**: applied on the processor's agent thread with no lock held while waiting (bounded 5 s); refused, as an `err` reply, for a stopped processor, a closed group or a full hand-off queue. Server stop closes every capture still recording.
- **Processor failures**: a processor that fails at runtime (added at runtime, stop/tearDown, a service (de)registration, a timer) is reported and isolated instead of exiting the JVM; a start failure **at boot still fails boot** by default (opt-in isolation: `isolateProcessorStartFailuresAtBoot`). `VirtualMachineError` still exits.
- **Control queues** hold 4096 items; a producer blocks up to 10 s when one is full (per group: `setControlQueueFullTimeout`); on the group's own thread a full queue fails at once.
- **Services** are delivered to each processor once.
- New API: `EventToQueuePublisher.droppedCount()`, `startComplete()`, `publishOrCache()`, `set/isRetainEventLogAfterStart`; `MongooseServer.awaitStopped()`, `isStarted()`, `processorDropped(...)`, `failsBootOnProcessorStartFailure()`; `EventFlowManager.releaseReader/readerCount`; `DeadWheelScheduler.cancelTimersOf`; `GroupReplayer.detach`; `MongooseAuditCaptureService.detach` (default no-op); `ProcessorStartException`; config `shutdownDrainTimeoutMillis`, `startupTimeoutMillis`, `isolateProcessorStartFailuresAtBoot`. Nothing removed.

mongoose-plugins 1.0.46

- svc-admin-rest: `POST /admin` works without adding Jackson; unknown fields → 400; unknown command → 404 (was an empty 200); no registry → 503; several replies are joined into one message; `/api` String replies are `text/plain`.
- svc-admin-web: a malformed body or unknown field → 400 (it used to run the command with no arguments); both `arguments` and `args` accepted.

5. Risks and mitigations

Risk	Mitigation
A user relied on post-start event-log history	`setRetainEventLogAfterStart(true)`; release note.
Duplicate processor names in YAML now fail boot	Release note; the error names the group and processor.
A slow subscriber throttles publishers on a shared pipe	Documented (`docs/architecture/event-flow.md`); watch `droppedCount()`.
Plugins' fluxtion 1.0.15 vs Mongoose's 1.1.x	The adoption PR bumps fluxtion-bom; run the reactor with tests.
A plugin implements `MongooseAuditCaptureService`	`detach` has a default no-op; nothing to change.
Stacked PRs merged out of order	Merge strictly in the stack order above; each PR's body names its base.